

BADT Assignment Saria Sato

Project Background

This project is one of the building blocks for a bigger project of my PhD work that looks into the effects of spring timing on the breeding success of an alpine bird, willow ptarmigan (*Lagopus lagopus*) in central Norway. As there is yearly variation in the number of chicks produced, we are interested in understanding if the yearly differences in greening /timing of spring affects this biological process. We have camera data from 2019 to 2025 where greenness is quantified by the number of green pixels in the picture. The camera image spans from mid-March to late-June (winter to spring transition) and are set up across 7 different plots within our study area.

For the purpose of BADT project, I focus on estimating the timing of greenness for each year. For each year, I am interested in first, estimating the mean greenness and then seeing the seasonal change in greenness for each of these years. I test three models to find which model best reflects the ecological process of greening and statistically explain my seasonal pattern well. I run a non-linear model by using a spline function and then use binomial distribution where n_{green} is the number of green pixels and N_{pixel} is trials, which is the total number of pixels.

NB: I did not use Year as random effect for two reasons

1. I would like to have year to year comparison and know which year was greener
2. I only have 7 years, which is a bit low to properly estimate stable variance

MODELS

- **Model binomial:** $n_{\text{green}} \mid \text{trials}(N_{\text{pixel}}) \sim \text{Year} + \text{s}(\text{JulianDay}, \text{by} = \text{Year}) + (1 \mid \text{Plot}),$
family = binomial (link = “logit”)
- **Model beta binomial with default priors:** $n_{\text{green}} \mid \text{trials}(N_{\text{pixel}}) \sim \text{Year} + \text{s}(\text{JulianDay}, \text{by} = \text{Year}) + (1 \mid \text{Plot})$
family = beta_binomial (link = “logit”)

- **Model beta-binomial with weakly informative prior:** $n_green \mid trials(N_pixel) \sim Year + s(JulianDay, by = Year) + (1|Plot)$
 $family = binomial(link = "logit")$ with the priors:
 $prior(normal(0, 1), class = "b")$
 $prior(normal(0, 2), class = "Intercept")$
 $prior(exponential(1), class = "sd")$
 $prior(exponential(1), class = "sds")$
 $prior(gamma(2, 0.1), class = "phi")$

Analysis

Step 1: Data Manipulation

Loading required package: Rcpp

Loading 'brms' package (version 2.23.0). Useful instructions can be found by typing `help('brms')`. A more detailed introduction to the package is available through `vignette('brms_overview')`.

Attaching package: 'brms'

The following object is masked from 'package:stats':

ar

This is cmdstanr version 0.9.0

- CmdStanR documentation and vignettes: mc-stan.org/cmdstanr

- CmdStan path: `/home/r3759939/.cmdstan/cmdstan-2.38.0`

- CmdStan version: 2.38.0

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v tibble    3.3.1
v lubridate  1.9.5      v tidyr     1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Attaching package: 'tidybayes'

The following objects are masked from 'package:brms':

dstudent_t, pstudent_t, qstudent_t, rstudent_t

```
# A tibble: 6 x 18
  Date          meanGreenFraction maxGreenFraction meanPeriodTempC
  <date>          <dbl>          <dbl>          <dbl>
1 2019-03-26      0.00162          0.163          -1
2 2019-03-27      0.00195          0.336           4
3 2019-03-28      0.00166          0.179          2.27
4 2019-03-29      0.00164          0.168         -0.273
5 2019-03-30      0.00339          2.01           1.36
6 2019-03-31      0.00172          0.239         -0.545
# i 14 more variables: medianPeriodTempC <dbl>, minPeriodTempC <dbl>,
#   maxPeriodTempC <dbl>, meanTempC <dbl>, medianTempC <dbl>, minTempC <dbl>,
#   maxTempC <dbl>, MemCard <chr>, Plot <fct>, JulianDay <int>, Year <fct>,
#   SourceFile <chr>, N_pixel <dbl>, n_green <int>
```

Step 2: Running Models

Model Binomial

Run the model

Loading required namespace: rstan

```
Family: beta_binomial
Links: mu = logit
Formula: n_green | trials(N_pixel) ~ Year + (1 | Plot)
```

Data: combined_data (Number of observations: 4648)
 Draws: 3 chains, each with iter = 2000; warmup = 500; thin = 1;
 total post-warmup draws = 4500

Multilevel Hyperparameters:

~Plot (Number of levels: 7)

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sd(Intercept)	0.20	0.08	0.11	0.40	1.00	1149	2016

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	-2.60	0.09	-2.78	-2.41	1.00	1483	1889
Year2020	-0.21	0.06	-0.32	-0.10	1.00	2363	2693
Year2021	-0.05	0.05	-0.15	0.06	1.00	2211	2606
Year2022	-0.20	0.06	-0.30	-0.08	1.00	2335	3057
Year2023	-0.02	0.06	-0.14	0.09	1.00	2180	2202
Year2024	0.22	0.06	0.11	0.33	1.00	1999	2962
Year2025	0.27	0.06	0.16	0.39	1.00	2258	2889

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
phi	5.03	0.13	4.77	5.29	1.00	3601	2853

Draws were sampled using `sample(hmc)`. For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

Results and Interpretation

Clearly, the binomial model failed to converge with R-hat value > 1. The model took 13 hours to run. Silly mistake but the lesson learned here is to check for overdispersion first and then to run the model. I switched to beta-binomial below and the model results improved.

Model Beta-Binomial with default priors

Run the model

```
Model_betabinomial <- readRDS("/cloud/project/Greeness/greening_M3.RDS")
summary(Model_betabinomial)
```

Warning: There were 7 divergent transitions after warmup. Increasing `adapt_delta` above 0.8 may help. See <http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup>

Family: beta_binomial
 Links: mu = logit
 Formula: n_green | trials(N_pixel) ~ Year + s(JulianDay, by = Year) + (1 | Plot)
 Data: combined_data (Number of observations: 4648)
 Draws: 3 chains, each with iter = 2000; warmup = 500; thin = 1;
 total post-warmup draws = 4500

Smoothing Spline Hyperparameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
sds(sJulianDayYear2019_1)	4.64	2.20	2.08	9.99	1.00	1911
sds(sJulianDayYear2020_1)	6.65	1.84	4.07	11.06	1.00	1975
sds(sJulianDayYear2021_1)	5.13	1.52	3.01	8.75	1.00	1889
sds(sJulianDayYear2022_1)	4.25	1.39	2.24	7.45	1.00	2199
sds(sJulianDayYear2023_1)	2.89	1.15	1.41	5.79	1.00	2633
sds(sJulianDayYear2024_1)	7.84	2.40	4.37	13.75	1.00	2337
sds(sJulianDayYear2025_1)	2.41	1.02	1.07	4.82	1.00	2706
	Tail_ESS					
sds(sJulianDayYear2019_1)	2132					
sds(sJulianDayYear2020_1)	2963					
sds(sJulianDayYear2021_1)	3024					
sds(sJulianDayYear2022_1)	2751					
sds(sJulianDayYear2023_1)	3031					
sds(sJulianDayYear2024_1)	2628					
sds(sJulianDayYear2025_1)	2645					

Multilevel Hyperparameters:

~Plot (Number of levels: 7)

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sd(Intercept)	0.57	0.22	0.30	1.14	1.00	1941	2812

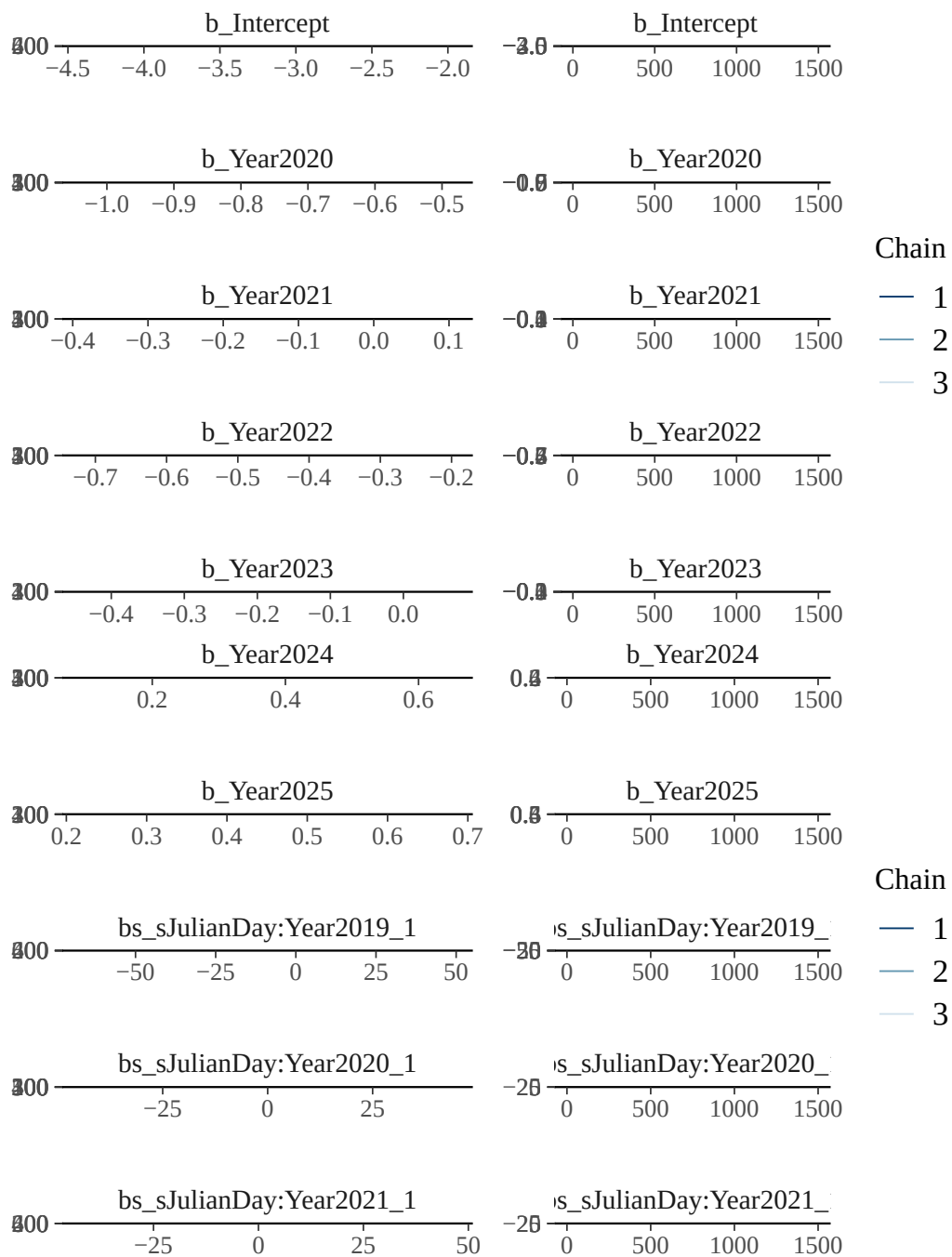
Regression Coefficients:

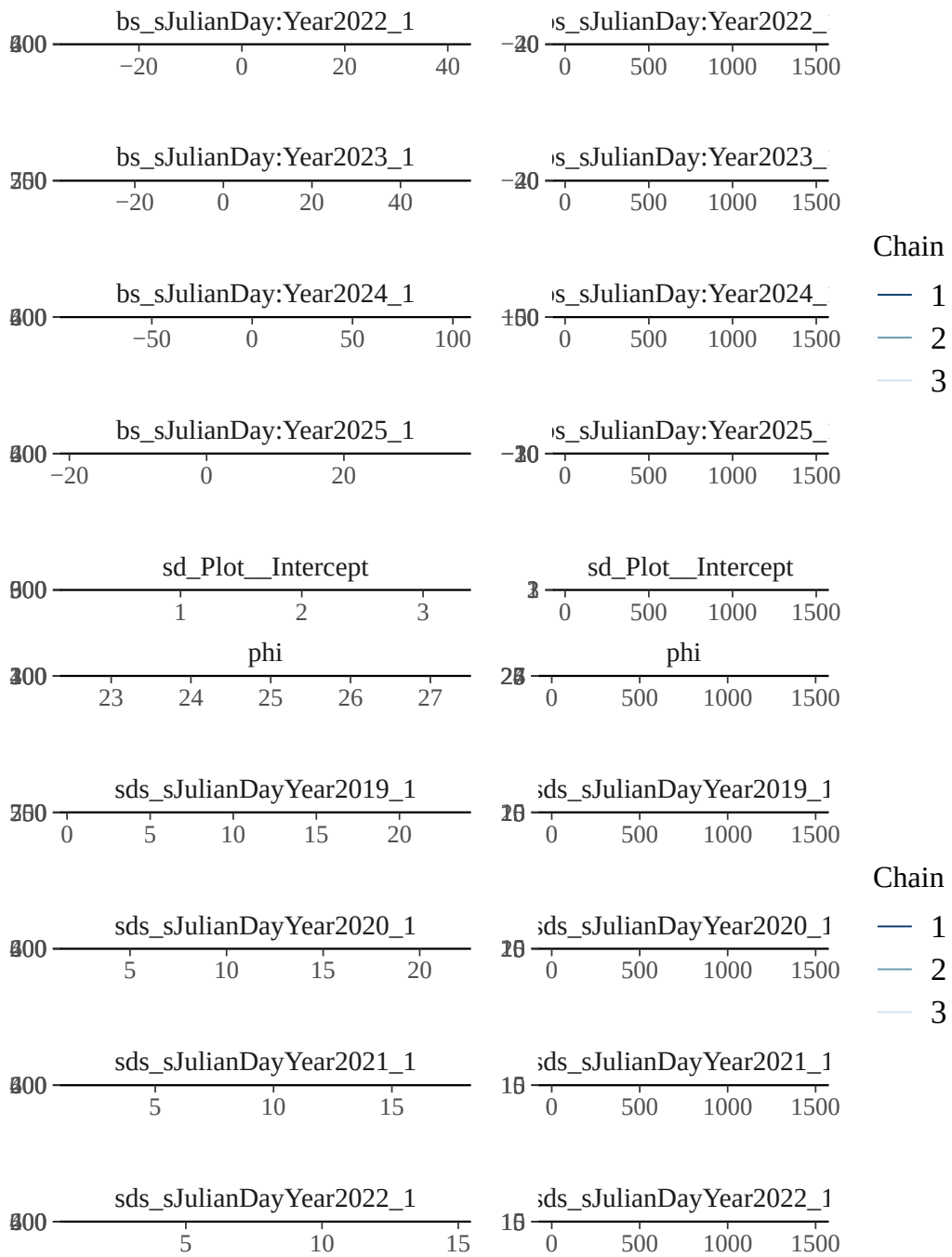
	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
Intercept	-3.21	0.23	-3.66	-2.73	1.00	1495
Year2020	-0.74	0.07	-0.87	-0.60	1.00	2101
Year2021	-0.15	0.06	-0.27	-0.03	1.00	1935
Year2022	-0.48	0.06	-0.59	-0.35	1.00	1840
Year2023	-0.21	0.06	-0.32	-0.08	1.00	1997
Year2024	0.33	0.07	0.20	0.46	1.00	1897
Year2025	0.43	0.06	0.32	0.56	1.00	1780
sJulianDay:Year2019_1	7.61	11.18	-18.53	27.39	1.00	2425
sJulianDay:Year2020_1	-4.76	9.66	-23.05	14.71	1.00	1866
sJulianDay:Year2021_1	0.22	8.17	-16.16	16.69	1.00	2200
sJulianDay:Year2022_1	4.93	8.22	-11.57	21.47	1.00	2245

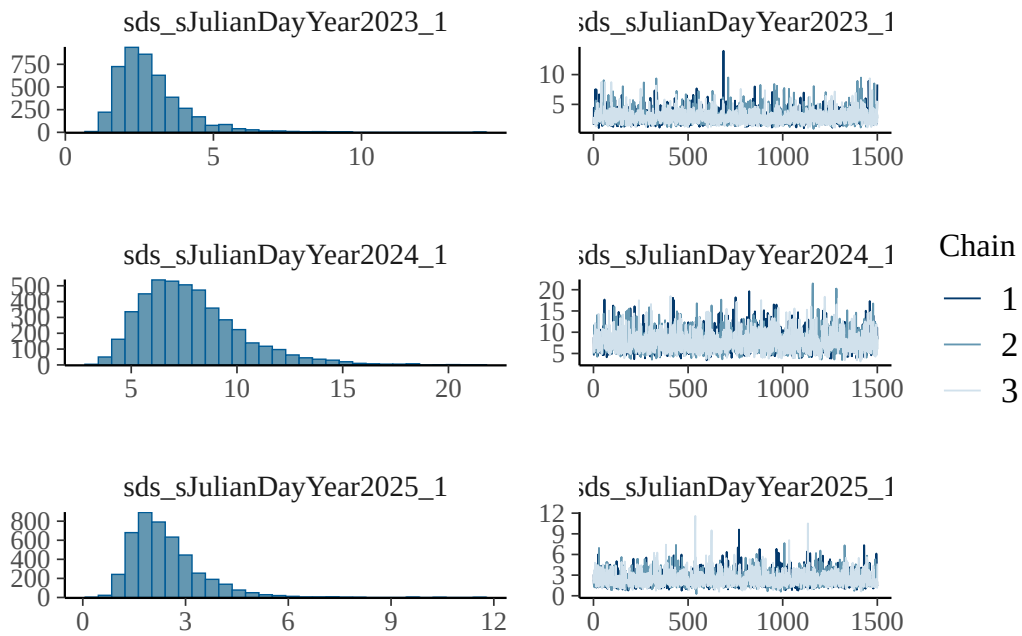
sJulianDay:Year2023_1	7.95	6.32	-4.96	20.62	1.00	2719
sJulianDay:Year2024_1	7.80	16.32	-23.62	41.00	1.00	1809
sJulianDay:Year2025_1	5.11	5.60	-6.60	16.33	1.00	3056
Tail_ESS						
Intercept	2276					
Year2020	2416					
Year2021	1969					
Year2022	2347					
Year2023	2126					
Year2024	2097					
Year2025	2002					
sJulianDay:Year2019_1	2076					
sJulianDay:Year2020_1	2050					
sJulianDay:Year2021_1	2268					
sJulianDay:Year2022_1	2354					
sJulianDay:Year2023_1	2755					
sJulianDay:Year2024_1	2375					
sJulianDay:Year2025_1	2956					

Further Distributional Parameters:

Draws were sampled using `sample(hmc)`. For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

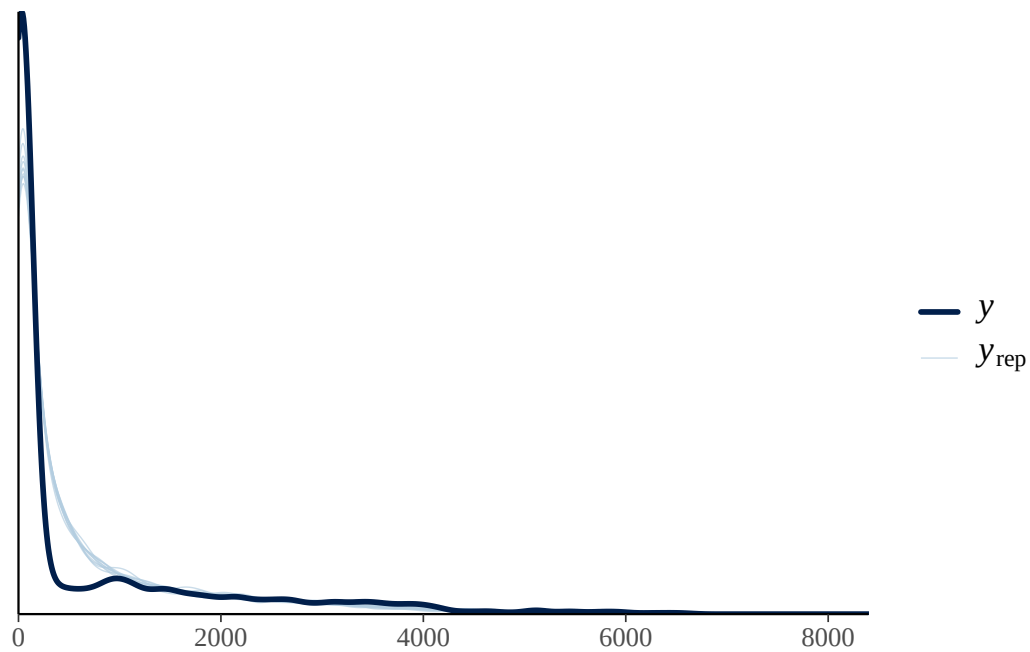






```
pp_check(Model_betabinomial)
```

Using 10 posterior draws for ppc type 'dens_overlay' by default.



Priors: here I used the default priors that is plausible for each parameter

```

get_prior(
  n_green | trials(N_pixel) ~ Year + s(JulianDay, by =Year) + (1 | Plot),
  data = combined_data,
  family = beta_binomial())

```

	prior	class	coef	group	resp	dpar	nlpar
	(flat)	b					
	(flat)	b	sJulianDay:Year2019_1				
	(flat)	b	sJulianDay:Year2020_1				
	(flat)	b	sJulianDay:Year2021_1				
	(flat)	b	sJulianDay:Year2022_1				
	(flat)	b	sJulianDay:Year2023_1				
	(flat)	b	sJulianDay:Year2024_1				
	(flat)	b	sJulianDay:Year2025_1				
	(flat)	b	Year2020				
	(flat)	b	Year2021				
	(flat)	b	Year2022				
	(flat)	b	Year2023				
	(flat)	b	Year2024				
	(flat)	b	Year2025				
student_t(3, 0, 2.5)		Intercept					
gamma(0.01, 0.01)		phi					
student_t(3, 0, 2.5)		sd					
student_t(3, 0, 2.5)		sd			Plot		
student_t(3, 0, 2.5)		sd	Intercept		Plot		
student_t(3, 0, 2.5)		sds					
student_t(3, 0, 2.5)		sds	s(JulianDay, by = Year)				
lb	ub	tag	source				
			default				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				
			(vectorized)				

```

                                default
0                                default
0                                default
0                                (vectorized)
0                                (vectorized)
0                                default
0                                (vectorized)

```

Interpret the beta-binomial model results

Model Validation and Fit: This is much better result compared to the first model with just binomial distribution. The Rhat value is 1 for each effect which indicates that the model converged well. The Bulk ESS and Tail ESS, which is our effective sample size are above 1000. The phi-value of 24.81 suggests that there is substantial overdispersion, which justifies our reason to use the beta-binomial distribution.

However, there are minor divergence issues in the model and more importantly the 95% CI and the standard error for the spline coefficient are also quite large. So I decided to put a weakly informative priors to set some biologically plausible values in hopes to improve the model performance. The model is presented below:

Model Beta-Binomial with weakly informative priors

Run the model

```

Model_3 <- readRDS("/cloud/project/BADT_Homework/Model_betabinomial_prior.rds")
summary(Model_3)

```

```

Family: beta_binomial
Links: mu = logit
Formula: n_green | trials(N_pixel) ~ Year + s(JulianDay, by = Year) + (1 | Plot)
Data: combined_data (Number of observations: 4648)
Draws: 3 chains, each with iter = 2000; warmup = 500; thin = 1;
       total post-warmup draws = 4500

```

Smoothing Spline Hyperparameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
sds(sJulianDayYear2019_1)	3.70	1.02	2.14	6.12	1.00	2734
sds(sJulianDayYear2020_1)	5.43	1.13	3.63	8.06	1.00	3142
sds(sJulianDayYear2021_1)	4.04	0.98	2.52	6.30	1.00	2514
sds(sJulianDayYear2022_1)	3.49	0.92	2.10	5.61	1.00	2642
sds(sJulianDayYear2023_1)	2.58	0.78	1.46	4.49	1.00	3169

sds(sJulianDayYear2024_1)	5.65	1.28	3.68	8.67	1.00	2729
sds(sJulianDayYear2025_1)	2.12	0.66	1.17	3.73	1.00	3705
	Tail_ESS					
sds(sJulianDayYear2019_1)	3485					
sds(sJulianDayYear2020_1)	3199					
sds(sJulianDayYear2021_1)	2907					
sds(sJulianDayYear2022_1)	2990					
sds(sJulianDayYear2023_1)	3377					
sds(sJulianDayYear2024_1)	2510					
sds(sJulianDayYear2025_1)	3422					

Multilevel Hyperparameters:

~Plot (Number of levels: 7)

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sd(Intercept)	0.53	0.19	0.29	1.00	1.00	1990	2831

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
Intercept	-3.20	0.21	-3.60	-2.77	1.00	1496
Year2020	-0.75	0.06	-0.87	-0.63	1.00	2392
Year2021	-0.16	0.06	-0.26	-0.05	1.00	2589
Year2022	-0.48	0.06	-0.59	-0.37	1.00	2679
Year2023	-0.20	0.06	-0.32	-0.09	1.00	2656
Year2024	0.32	0.06	0.21	0.44	1.00	2452
Year2025	0.43	0.06	0.32	0.54	1.00	2498
sJulianDay:Year2019_1	0.16	1.00	-1.79	2.04	1.00	10776
sJulianDay:Year2020_1	-0.12	1.03	-2.10	1.88	1.00	8430
sJulianDay:Year2021_1	0.03	0.97	-1.87	1.92	1.00	9812
sJulianDay:Year2022_1	0.14	0.99	-1.75	2.08	1.00	10446
sJulianDay:Year2023_1	0.31	0.99	-1.67	2.20	1.00	11275
sJulianDay:Year2024_1	0.04	0.97	-1.87	1.94	1.00	11617
sJulianDay:Year2025_1	0.29	0.99	-1.61	2.14	1.00	10042
	Tail_ESS					
Intercept	2179					
Year2020	3426					
Year2021	3286					
Year2022	3331					
Year2023	3526					
Year2024	3007					
Year2025	2983					
sJulianDay:Year2019_1	3150					
sJulianDay:Year2020_1	3254					
sJulianDay:Year2021_1	3017					

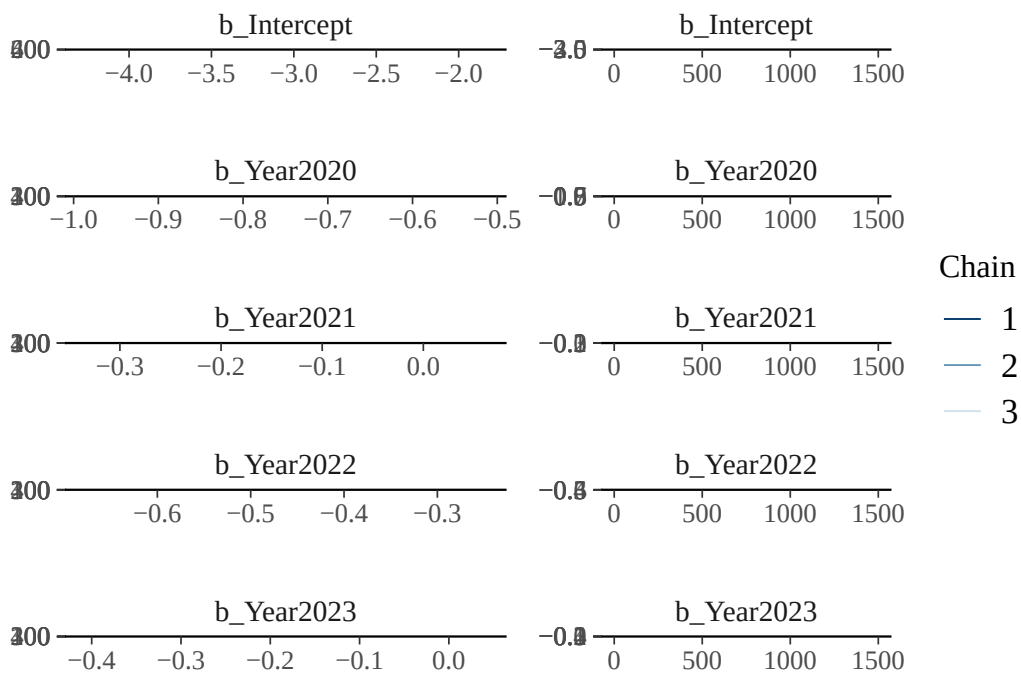
```
sJulianDay:Year2022_1      3213
sJulianDay:Year2023_1      3073
sJulianDay:Year2024_1      3275
sJulianDay:Year2025_1      3194
```

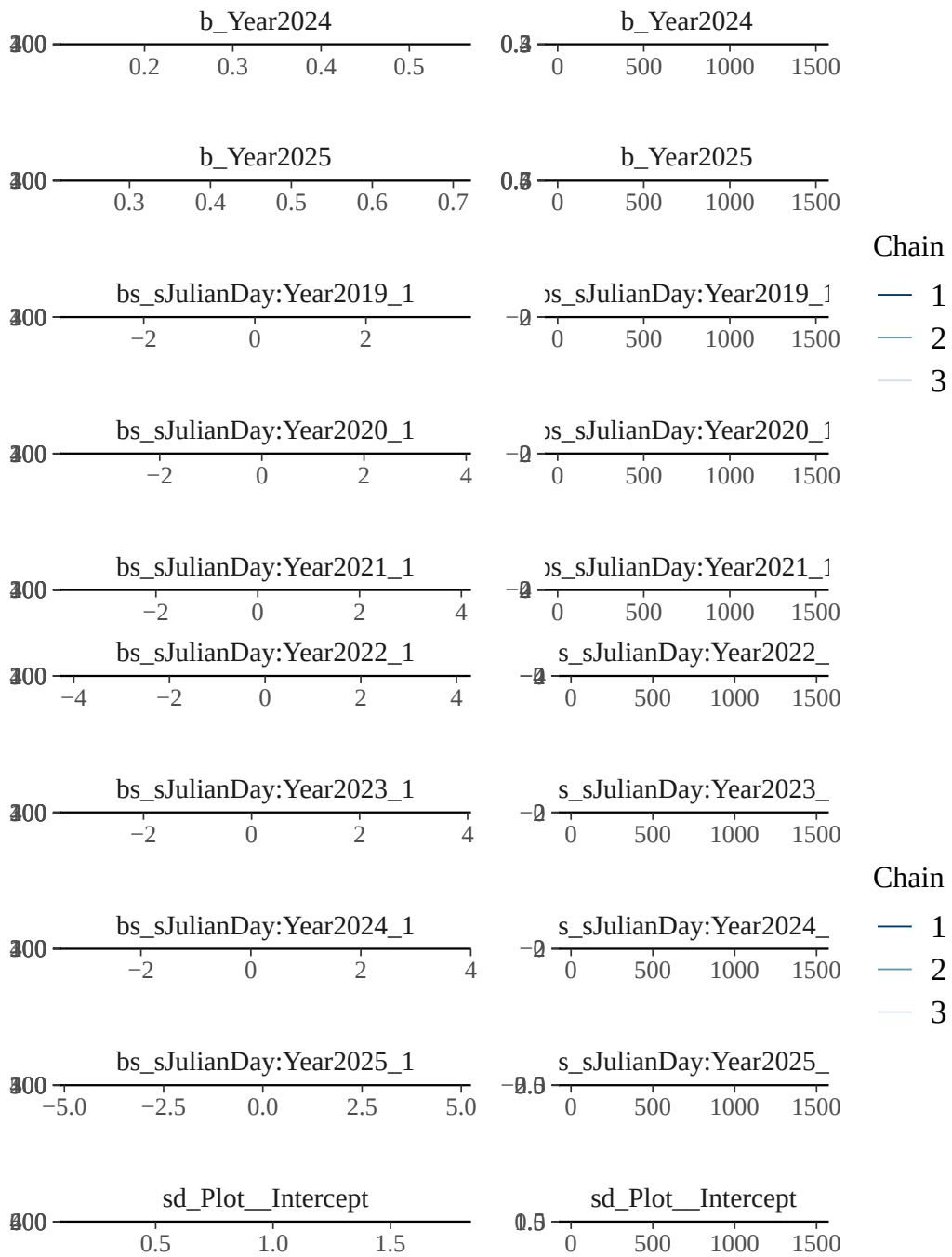
Further Distributional Parameters:

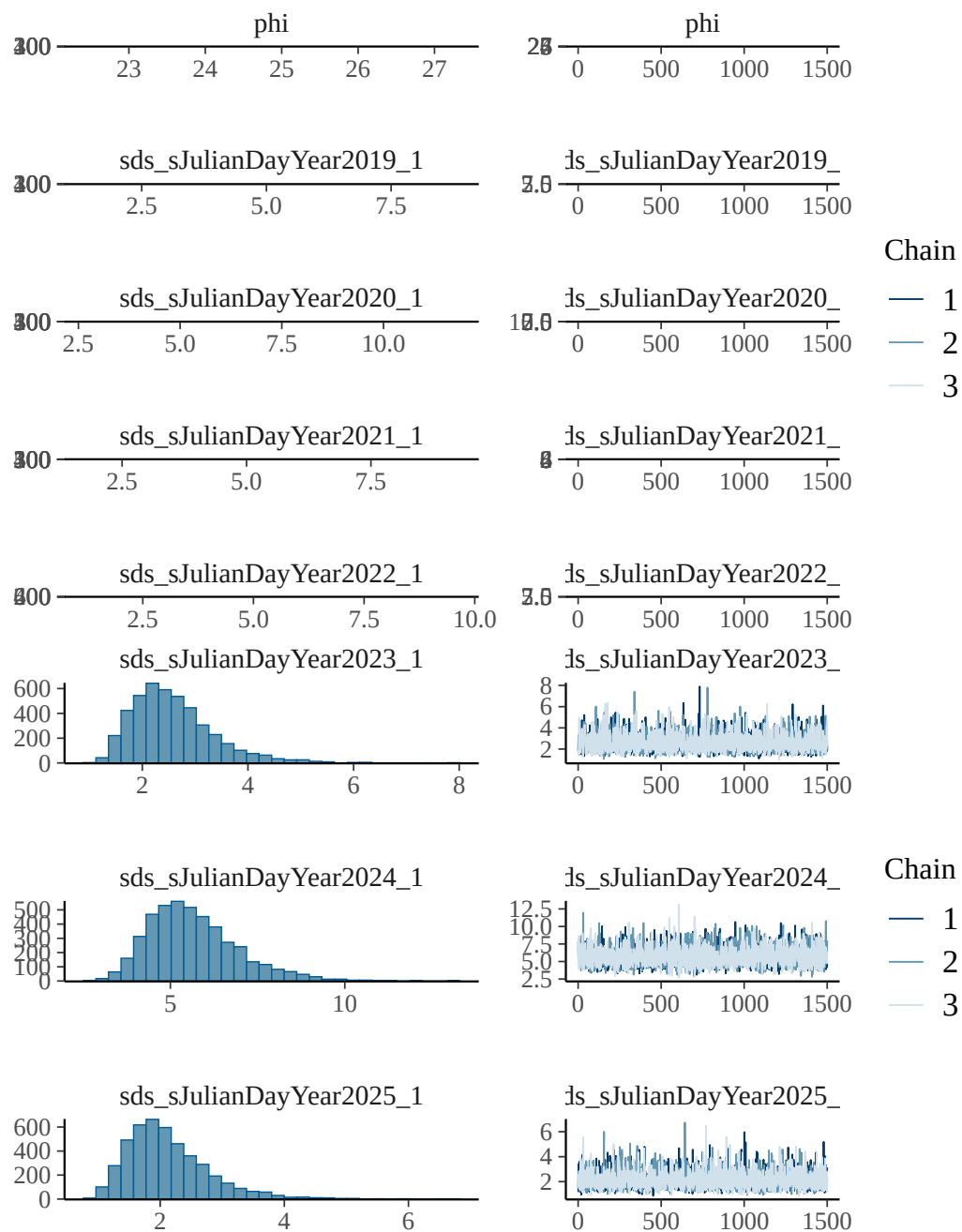
	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
phi	24.81	0.66	23.52	26.17	1.00	11306	2994

Draws were sampled using `sample(hmc)`. For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

```
plot(Model_3)
```

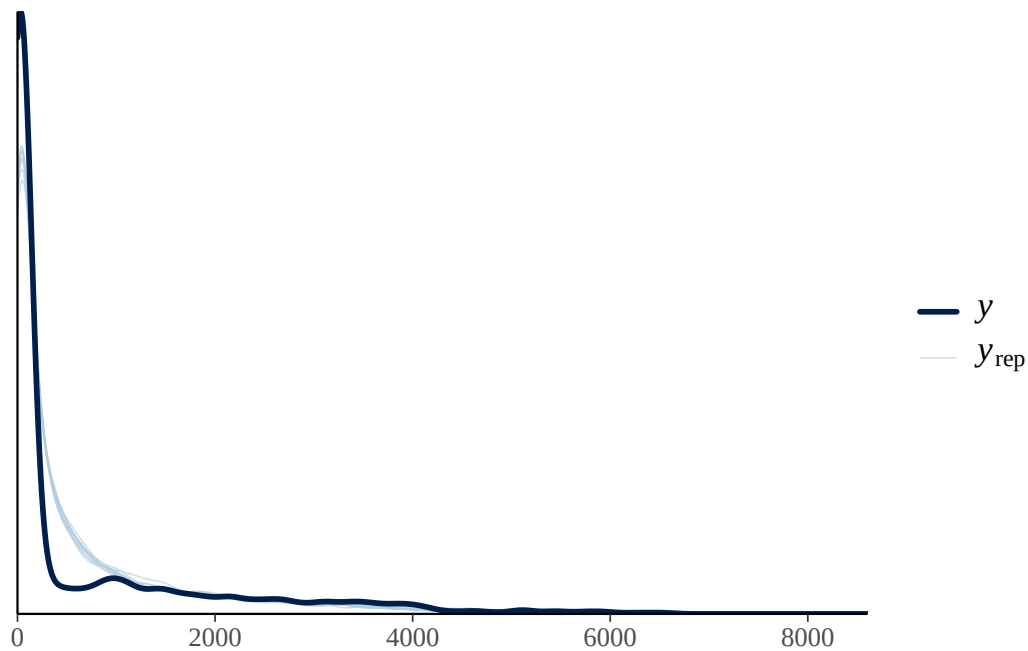






`pp_check (Model_3)`

Using 10 posterior draws for ppc type 'dens_overlay' by default.



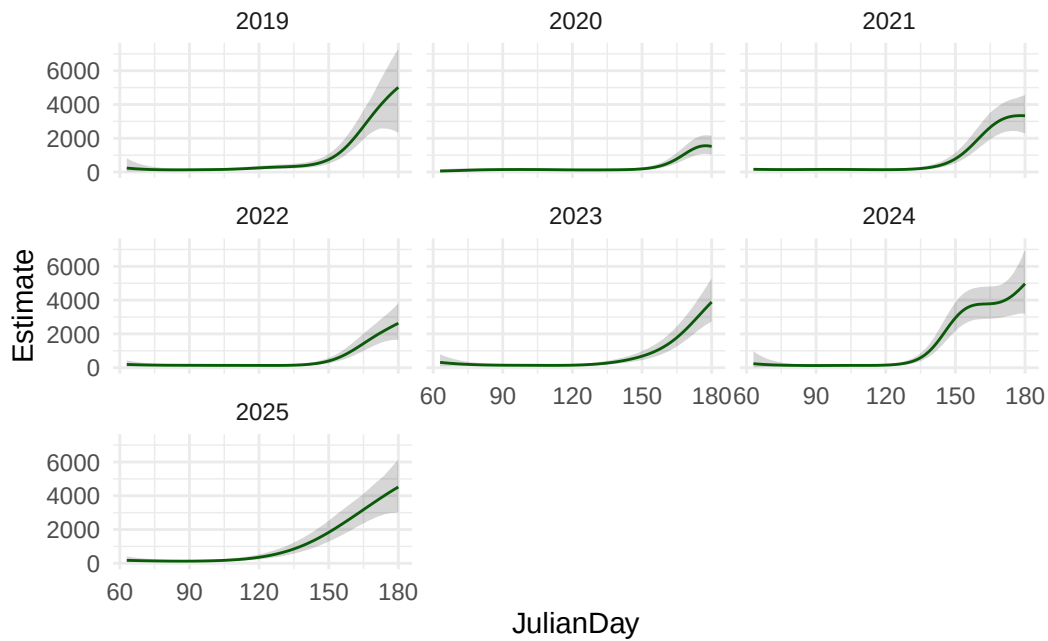
Plot the seasonal trend of greening for each year

```
newdata <- expand.grid(
  JulianDay = seq(min(combined_data$JulianDay),
    max(combined_data$JulianDay), by = 1),
  Year = levels(combined_data$Year),
  Plot = NA
)
newdata$N_pixel <- 10000
pred <- fitted(Model_3, newdata = newdata, re_formula = NA)

plot_data <- cbind(newdata, pred)
plot_data_trim <- plot_data %>%
  dplyr::filter(JulianDay <= 180)

p <- ggplot(plot_data_trim, aes(x = JulianDay, y = Estimate)) +
  geom_line(color = "darkgreen") +
  geom_ribbon(aes(ymin = Q2.5, ymax = Q97.5), alpha = 0.2) +
  facet_wrap(~Year) +
  theme_minimal()

print(p)
```

Extract posterior predictions to find the timing of spring

```
##### Extract the posterior predictions #####
posterior <- posterior_epred(
  Model_3,
  newdata = newdata,
  re_formula = NA
)

### Convert to proportions and create a function to loop through the posterior draws ##
posterior_prop <- posterior / newdata$N_pixel

spring_draws <- apply(posterior_prop, 1, function(draw_values) {

  df <- newdata
  df$prop <- draw_values

  df %>%
    group_by(Year) %>%
    summarise(
      spring_day = {
        valid_days <- JulianDay[prop > 0.1 & !is.na(prop)]
        if (length(valid_days) == 0) NA else min(valid_days)
      },

```

```

      .groups = "drop"
    ) %>%
    pull(spring_day)
  })

### Then Convert to dataframe to summarize ###
spring_draws_df <- as.data.frame(t(spring_draws))
colnames(spring_draws_df) <- levels(newdata$Year)

spring_summary <- spring_draws_df %>%
  summarise(across(everything(), list(
    mean = ~mean(., na.rm = TRUE),
    lower = ~quantile(., 0.025, na.rm = TRUE),
    upper = ~quantile(., 0.975, na.rm = TRUE)
  )))

spring_summary_long <- spring_summary %>%
  tidyr::pivot_longer(
    everything(),
    names_to = c("Year", ".value"),
    names_sep = "_"
  )

spring_summary_long

```

```

# A tibble: 7 x 4
  Year    mean lower upper
<chr> <dbl> <dbl> <dbl>
1 2019   153.   149   158
2 2020   168.   164   173
3 2021   153.   149   157
4 2022   161.   156   166
5 2023   156.   150   162
6 2024   138.   134   142
7 2025   139.   132   146

```

Plot the distribution

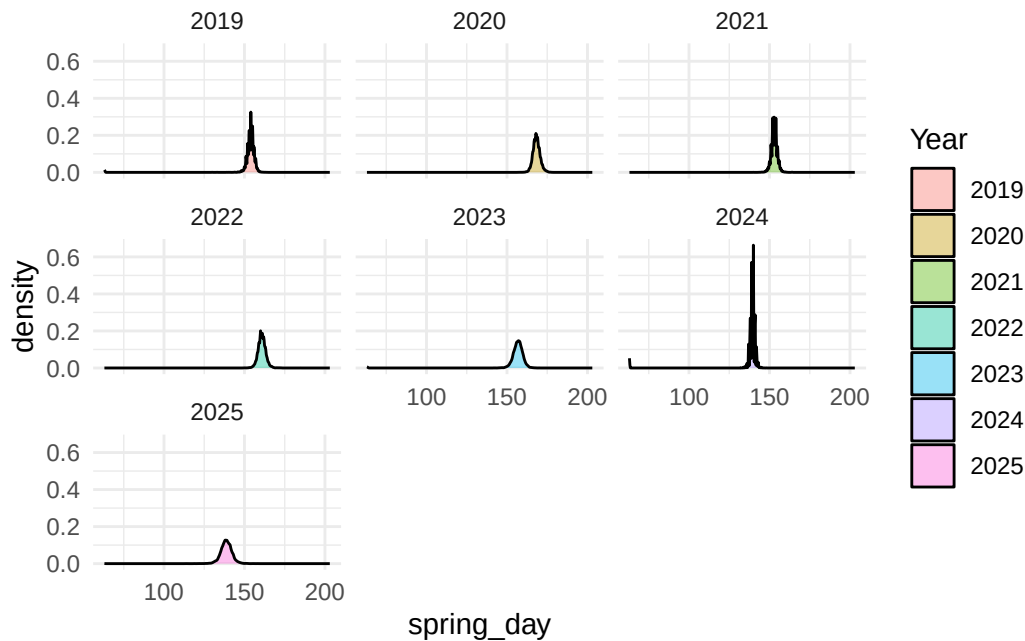
```

spring_draws_df %>%
  pivot_longer(everything(), names_to = "Year", values_to = "spring_day") %>%
  ggplot(aes(x = spring_day, fill = Year)) +

```

```
geom_density(alpha = 0.4) +
facet_wrap(~Year) +
theme_minimal()
```

Warning: Removed 59 rows containing non-finite outside the scale range (`stat\_density()`).



Results

Compared to the model with uninformative default priors, the model with weakly informative priors I set performed much better. Firstly the spline coefficient (sds smoothness) greatly improved. For example in year 2019, sds (sJulianDayYear2019_1) was 4.64 (2.08- 9.99). But with the prior, exponential (1) on sds, the sds (sJulianDayYear2019_1) was lowered to 3.7 (2.14- 6.12). The stronger prior helped to smoothen the curve and make it more realistic. Yet, the regression coefficient, random effect and dispersion (phi) values remained almost all identical despite setting stronger priors on these parameters as well. The ESS values also increased meaning the sampling quality got better with the priors I set. In conclusion, this model with the weakly informative prior improved the smoother and provided more stable estimates of seasonal greenness patterns in each year.

In terms of answering my own scientific question about the mean greenness each year and the timing of greenness each year, both models provided very similar values. My preliminary

analysis shows a strong evidence for the yearly variation in average greenness levels between 2019 to 2025. In particular, the regression coefficient indicate that the year 2020 was the year with lowest greenness level ($\beta = -0.74$ with 95 % CI = - 0.87, - 0.63) and the years 2024 and 2025 had the most greenness ($\beta = 0.32$ - 0.43) in relation to the year 2019.

There is some variation in greenness (SD = 0.53) across the plots which indicates that there is spatial heterogeneity within our study area. This is expected as (i) some plots are more south-facing compared to the others (ii) plots have different types of vegetation. The model has accounted for this random effect well, given the big ESS and Rhat value of 1.

As to answer the question when does spring/greening actually start for each year, I decided that spring begins when the proportion of greenness goes above 0.1 (this is an arbitrary value and I am yet to figure out the best way to define the threshold but I am thinking to use X% of the maximum greenness or maximum slope for example). Using this value, I extracted the Julian Day for each year to decide the day of spring (results in the table with 95% Credible Interval). Spring was late in 2020 (Julian day 168), which also aligns well with the lowest greenness value and the weather information known about the study area during that year. It seems 2025 and 2024 was the year with earliest springs.

While this is a preliminary investigation of the greenness data, I am continuing to explore and optimize more model structures together with finding perhaps a better estimation criteria for the timing of spring. I will eventually use LOO to rank and compare the models. The effect and timing of greenness on the breeding success will then be explored after this. But for the purpose of this project and the given time frame, I will conclude the work here.